

Database Systems I / Information Systems I

Transaction Management — Complementary Material

Christian Rauch

(changed: 2026-06-29)

What is a Transaction?

A **transaction** is a sequence of database operations that must be executed as a single logical unit of work.

Classic example – bank transfer:

1. Debit \$500 from account A.
2. Credit \$500 to account B.

Both steps must succeed together. If step 2 fails after step 1, the database would be left in an **inconsistent state** – money vanishes.

Solution

The DBMS guarantees that a transaction either **commits** (all changes take effect) or **aborts** (all changes are undone). There is no partial success.

ACID Properties

Every transaction must satisfy four guarantees:

Atomicity All operations in the transaction succeed, or none of them do. A crash mid-transaction leaves no partial changes.

Consistency A transaction takes the database from one *valid* state to another. All integrity constraints hold before and after.

Isolation Concurrent transactions do not interfere with each other. Each transaction executes as if it were the only one running.

Durability Once a transaction commits, its changes survive any subsequent failure – power loss, crash, etc.

Responsibility

Atomicity & Durability – enforced by the *recovery manager*.

Consistency – enforced by integrity constraints + application.

Isolation – enforced by the *concurrency manager*.

BEGIN, COMMIT, ROLLBACK

SQL exposes transaction control through three statements:

BEGIN Marks the start of a transaction. All subsequent statements belong to it until explicitly ended.

COMMIT Makes all changes **permanent** and durable. The DBMS guarantees they survive any subsequent failure.

ROLLBACK **Undoes** all changes since BEGIN. Restores the database to the state before the transaction started.

Typical structure

```
BEGIN TRANSACTION;
```

```
-- database operations ...
```

```
COMMIT;    (or ROLLBACK; on error / abort)
```

Savepoints

A **savepoint** marks a named recovery point *inside* a running transaction.

Commands

SAVEPOINT *name* – mark the current position.

ROLLBACK TO *name* – undo all work *since* that savepoint, without aborting the whole transaction.

RELEASE SAVEPOINT *name* – discard the savepoint (changes are kept; the mark is just removed).

Use case

Long transactions with independent subtasks: if subtask B fails, roll back only to the savepoint after A committed its local work – no need to redo A from scratch.

Types of Failures

The recovery manager must handle three distinct failure categories:

Transaction failure A single transaction aborts – either by explicit ROLLBACK, an integrity constraint violation, or a deadlock resolution. The database is otherwise intact; only this transaction's changes must be undone.

System failure The DBMS process crashes (OS crash, power loss, hardware fault). The contents of **main memory are lost**; disk data survives. Committed transactions that were still in memory buffers may not yet be on disk.

Media failure A disk (or storage medium) fails and its data is permanently lost. Requires **archival backups** and possibly remote replicas. Far rarer but most expensive to recover from.

Why System Failures Are Tricky

After a crash, the database may be in an **inconsistent state** for two distinct reasons:

Uncommitted changes on disk (Dirty writes)

A transaction was in progress when the crash occurred. Some of its writes may have been flushed to disk by the buffer manager before the transaction committed. These partial changes must be **undone** (*UNDO*).

Committed changes not yet on disk (Lost writes)

A transaction committed successfully, but its changes were still in memory buffers and not yet written to disk when the crash happened. These changes must be **redone** (*REDO*).

The recovery algorithm must identify which transactions need UNDO and which need REDO – this is the role of the **write-ahead log (WAL)**.

Storage Primitives (1/2) – Disk ↔ Buffer

The recovery model is built on four atomic operations across three layers: **Disk** ↔ **Buffer** ↔ **Program variable**.

Input(X) Read the disk block containing X into a **main-memory buffer block**.

No-op if the block is already buffered.

Output(X) Write the buffer block containing X back to **disk**, making the change durable.

Triggered by buffer eviction or forced by the recovery manager.

Key point

Input/Output move entire *disk blocks*. They are controlled by the **buffer manager**, not the application.

Storage Primitives (2/2) – Buffer $\leftrightarrow$ Variable

Read(X, x) Copy the value of X from the buffer into **program variable** x .

Calls **Input(X)** first if the block is not yet buffered.

Write(X, x) Copy x into the buffer block holding X .

Does **not** call **Output** – the page is marked *dirty* and stays in the buffer until explicitly flushed.

Why **Write** $\neq$ **Output** – the two problems

- ▶ Committed T called **Write** but **Output** never ran: changes are in the buffer, *not* on disk. $\Rightarrow$ **REDO** needed after a crash.
- ▶ Uncommitted T had its dirty buffer block evicted (**Output**) by the buffer manager under memory pressure. $\Rightarrow$ **UNDO** needed after a crash.

The Write-Ahead Log – Log Records

The **log** is an append-only sequence of records on *stable storage* that narrates the full life of every transaction.

Three record types

Event	Log record
Transaction T begins	$\langle T \text{ start} \rangle$
T modifies item X	$\langle T, X, \text{old: } v, \text{new: } w \rangle$
T commits	$\langle T \text{ commit} \rangle$

Example: bank transfer (debit A by \$500, credit B by \$500)

$\langle T_1 \text{ start} \rangle$
 $\langle T_1, A, \text{old: } 1000, \text{new: } 500 \rangle$
 $\langle T_1, B, \text{old: } 200, \text{new: } 700 \rangle$
 $\langle T_1 \text{ commit} \rangle$

The *old* value enables **UNDO**; the *new* value enables **REDO**.
Both must be logged before the data page can be written to disk.

The Write-Ahead Log – Rule 1: Write-Ahead

Write-ahead rule (WAL)

The log record $\langle T, X, old, new \rangle$ must reach stable storage **before** the data page containing X is written to disk.

Correct order:

1. Write log record to stable storage.
2. Write data page to disk.

After a crash: log record present $\Rightarrow$ can UNDO if T never committed.

Wrong order:

1. Write data page to disk.
2. *(crash here)*

Change is on disk; log has no record of it. Cannot UNDO. Database is **silently corrupted**.

Practical implication

The buffer manager must flush the log record to stable storage *before* evicting any dirty page that was modified by T .

The Write-Ahead Log – Rule 2: Commit Rule

Commit rule

Transaction T is **committed** only when $\langle T \text{ commit} \rangle$ has been flushed to stable storage – after *all* preceding log records for T .

Sufficient – flushing it is enough:

Once the commit record is on stable storage, a crashed system can REDO T entirely from the log – even if *none* of T 's data pages had been flushed yet. No further action is needed at commit time.

Necessary – flushing it is required:

Without the commit record on stable storage, the system has no proof that T ever committed. After a crash it will incorrectly **UNDO** T , silently discarding committed work.

Key insight

Data pages do *not* need to be on disk at commit time. COMMIT just means: log record flushed. Pages follow lazily.

The Write-Ahead Log – Architecture

Two subsystems handle data and log independently:

Buffer (Cache) Manager – data path

Moves **data pages** between disk and the buffer pool. Writes dirty pages asynchronously – driven by buffer pressure, not by transaction boundaries. Does not know when T commits.

Recovery Manager – log path

Appends **log records** to stable storage. Does not control when data pages are flushed.

The two paths are **decoupled** – coordinated only by the two WAL rules:

- ▶ log record before data page hits disk (Rule 1)
- ▶ commit record before declaring commit (Rule 2)

Why stable storage for the log?

A log in memory vanishes on crash. Stable storage (dedicated disk, battery-backed NVRAM) survives power loss – the log persists even when main memory does not.

Log Buffering

Writing every log record directly to stable storage is safe but costly – each append becomes a synchronous disk write. **Log buffering** relaxes this: records accumulate in a memory buffer and are flushed in batches.

Performance gain

Many log records are written to stable storage in a single I/O instead of one I/O per record. Throughput increases significantly under high transaction rates.

Despite batching, the same correctness guarantees must hold. See next slide for the three rules that must still be respected.

Log Buffering – Three Rules

1. Commit rule

$\langle T \text{ commit} \rangle$ must be flushed to stable storage **before** T is declared committed. A transaction cannot commit while its commit record sits only in the log buffer.

2. Ordering rule

Before $\langle T \text{ commit} \rangle$ is flushed, *all* earlier log records for T must already be on stable storage. The commit record is meaningless without the full history of T 's writes.

3. Write-ahead rule

Before a *data page* is written to disk, all log records that describe changes to that page must first be flushed to stable storage. (Same as the WAL rule – still required even when the log is buffered.)

Recovery Strategies – Overview

How and when data pages are written to disk determines which log values are needed and how complex recovery becomes.

Strategy	When pages hit disk	Log needs	Recovery
Undo/Redo	Any time (async) – buffer manager has full freedom	Before + after values	UNDO + REDO
No-Undo/Redo	Only <i>after</i> commit record flushed (deferred)	After values only	REDO only
Undo/No-Redo	All <i>before</i> commit (forced flush)	Before values only	UNDO only
No-Undo/No-Redo	Shadow paging: writes like Undo/Redo but all flushed before commit record	No values needed	None

The choice trades **normal-operation performance** against **recovery complexity**: fewer log values mean simpler recovery but more constraints on when the buffer manager may flush pages.

Undo/Redo – Immediate Modification

How it works

Modified pages may be written to disk at *any time* – before or after commit – as long as the corresponding log record has been flushed first (write-ahead rule). Both *before* and *after* values are recorded in the log.

Trade-off

Maximum flexibility for the buffer manager.

Most complex at recovery time: both UNDO and REDO must be implemented.

Large numbers of transactions may need to be redone after a crash because it is unknown which pages were on disk at failure time.

Undo/Redo – Recovery Procedure

After a crash

1. **Redo** every transaction that has both $\langle T \text{ start} \rangle$ and $\langle T \text{ commit} \rangle$ in the log – its pages may not yet be on disk.
2. **Undo** every transaction that has $\langle T \text{ start} \rangle$ but no $\langle T \text{ commit} \rangle$ – partial writes may already be on disk.

Idempotency requirement

If the system crashes a second time *during* recovery, the recovery procedure must be safe to restart from scratch and still produce a correct result. This is guaranteed by replaying the log in a fixed order and using log sequence numbers to skip already-applied records.

No-Undo/Redo – Deferred Modification

How it works

Pages are **not written to disk until after** the commit log entry is on stable storage. All writes go to the log and to the database cache only. After commit, the cached values are flushed to disk at leisure. Only *after* (new) values need to be logged – there is nothing to undo because no uncommitted data ever reaches disk.

Advantages

- ▶ Recovery requires only REDO – simpler and faster.
- ▶ No before values needed, so the log is smaller.

Disadvantages

- ▶ All dirty pages must be withheld from disk until commit – hot data occupies buffer space longer.
- ▶ Commit itself is more expensive: the system must ensure no modified page has been prematurely flushed.

Undo/No-Redo – Force on Commit

How it works

All pages modified by a transaction are **flushed to disk before the commit record is written**. The commit log entry is written only after every dirty page is safe on disk. Only *before* (old) values are needed in the log – committed data is already on disk, so nothing need be redone.

Advantages

- ▶ Recovery requires only UNDO – simpler.
- ▶ No after values in the log.

Disadvantages

- ▶ Every commit forces a flush of all dirty pages – high I/O traffic, especially for frequently updated (*hot-spot*) data items.

No-Undo/No-Redo – Shadow Paging

How it works

A transaction never overwrites live database pages. Changes are written to *shadow pages* at new disk locations. At commit, a single atomic pointer swap in the page directory makes the shadow pages the new live pages – one disk write. Abort simply discards the shadow pages; the live pages are untouched.

Advantages

- ▶ Recovery is instantaneous: no log to scan, no undo or redo.
- ▶ No log values (before or after) are needed at all.

No-Undo/No-Redo – Shadow Paging (Drawbacks)

Disadvantages

- ▶ **Destroyed locality:** pages are scattered across disk after each commit; sequential access patterns are lost.
- ▶ **Garbage collection:** superseded shadow pages accumulate and must be reclaimed periodically.
- ▶ **Concurrency:** the atomic pointer swap is a global operation; supporting concurrent transactions requires significant extra mechanism.
- ▶ **Indirect access:** all page references go through the page directory, adding a level of indirection to every read.

These drawbacks make shadow paging impractical for general-purpose DBMS workloads; it is mainly of theoretical interest.

Checkpointing – Procedure

Without checkpoints, recovery must scan the *entire* log – impractical as logs grow over time. A **checkpoint** captures a known consistent on-disk state; recovery only needs to look at the log from that point onward.

Steps (performed periodically during normal operation)

1. Flush all **log buffers** to stable storage.
2. Force all **dirty data pages** from the cache to disk.
3. Append $\langle checkpoint \rangle$ to the log.

No new transactions are admitted while steps 1–3 execute.

After step 3, every page modified before the checkpoint is safely on disk – they need neither redo nor undo during recovery.

Checkpointing – Recovery

Recovery procedure after a crash

1. Find the most recent $\langle checkpoint \rangle$ in the log.
2. **Redo** every transaction that committed *after* the checkpoint: its pages were not yet forced to disk at checkpoint time.
3. **Undo** every transaction that had started but not yet committed at or after the checkpoint: it may have left partial writes on disk.

Why this is correct

All pages modified *before* the checkpoint are already on disk (step 2 of the checkpoint procedure forced them). Only work done *after* the checkpoint can be missing or partial – the log covers exactly that window.

Concurrency and Scheduling

Model: centralized system with concurrent access by several users.

Schedule A sequence of operations from one or more transactions. In concurrent transactions, the operations of different transactions are *interleaved*.

Operations

- ▶ $\text{read}(Q, q)$ – read database item Q into local variable q .
- ▶ $\text{write}(Q, q)$ – store local variable q into database item Q .
- ▶ Arithmetic and other local operations.
- ▶ **Commit** – make all changes permanent.
- ▶ **Rollback** – undo all changes.

Example Schedule: T1 and T2

Two transactions transfer money between accounts A and B :

T1 ($A \rightarrow B$, \$50)

1. read(A , a)
2. $a \leftarrow a - 50$
3. write(A , a)
4. read(B , b)
5. $b \leftarrow b + 50$
6. write(B , b)

Interleaved schedule

T1

T2

read(A , a)
 $a \leftarrow a - 50$
write(A , a)

read(A , a)
 $a \leftarrow a \times 1.1$
write(A , a)

read(B , b)
 $b \leftarrow b + 50$
write(B , b)

T2 reads the value of A *already modified* by T1.

Serializability

Interleaving transactions can produce **incorrect results** even when each transaction is individually correct.

Three classic anomalies:

Lost Update T1 and T2 both read X , both modify it, and T1's update is overwritten by T2 – T1's change is lost.

Dirty Read T2 reads a value written by T1 before T1 commits. If T1 later aborts, T2 has used data that never existed.

Unrepeatable Read T1 reads X , T2 modifies and commits X , then T1 reads X again and gets a different value.

Goal: Serializability

A concurrent schedule is **correct** if its effect is equivalent to *some* serial execution of the same transactions. Serial schedules are trivially correct – no interleaving occurs.

Determining Serializability (1/2)

Conflicting operations

T_i and T_j **conflict** on item X iff both access X and *at least one* of them writes X . A conflict **fixes an execution order**: swapping the two operations changes the observable result.

Conflict equivalence via swapping:

Two constraints are always in force:

- ▶ **Intra-transaction order is fixed.** Operations within the same transaction are never reordered.
- ▶ **Only inter-transaction swaps are considered.** If two *consecutive* instructions $I \in T_i$ and $J \in T_j$ ($i \neq j$) do *not conflict*, swapping them yields a new schedule S' with identical observable effect. S and S' are **conflict equivalent**.

Determining Serializability (2/2)

Conflict serializability

S is **conflict serializable** iff $\exists$ a serial schedule S_{serial} that is conflict equivalent to S .

The swap argument defines conflict equivalence but is not a usable algorithm – the number of possible swap sequences grows exponentially. The **conflict graph** (next slides) provides an efficient test: a schedule is conflict serializable **if and only if** its conflict graph is acyclic.

No conflict – can swap:

T1	T2
read(A)	
	read(B)
write(A)	
	write(B)

$A \neq B$: the two middle steps touch different items. Either order yields the same result.

Conflict – cannot swap:

T1	T2
read(A)	
	write(A)
write(A)	

T1 reads A, T2 writes A – one writes, same item. Swapping T1's read past T2's write changes what T1 sees.

Possible Transaction Conflicts

For two transactions T_1 and T_2 accessing item X :

Write/Read T_1 writes X , then T_2 reads X . T_2 must execute after T_1 – it depends on T_1 's value.

Read/Write T_1 reads X , then T_2 writes X . T_2 must execute after T_1 – T_1 must see the old value.

Write/Write T_1 writes X , then T_2 writes X . T_2 must execute after T_1 – otherwise T_1 's write is lost.

Read/Read *No conflict* – both transactions read the same value of X ; no ordering is implied.

Conflict Graph

A **conflict graph** (also: precedence graph) depicts all conflicts in a schedule.

Construction:

- ▶ Assumption: no *blind writes* (every write is preceded by a read).
- ▶ One node per transaction.
- ▶ Draw a directed edge labelled X from T_i to T_j if T_i conflicts with T_j on item X and T_i 's conflicting operation appears before T_j 's.

Serializability test

A schedule is conflict serializable **if and only if** its conflict graph is **acyclic**. A topological sort of the graph yields an equivalent serial order.

Conflict Graph – Lock-Based Definition

When transactions use locks, the edge condition can be stated directly in lock terms.

Precedence edge $T_i \rightarrow T_j$

Draw a directed edge from T_i to T_j if there exists an item Q such that:

1. T_i has held lock mode A on Q , and T_j has held lock mode B on Q .
2. T_j 's lock on Q is *later* than T_i 's lock on Q .
3. At least one of A , B is an **exclusive** lock.

An edge $T_i \rightarrow T_j$ means T_i **must precede** T_j in any equivalent serial schedule – their lock modes on the shared item are incompatible and T_i acted first.

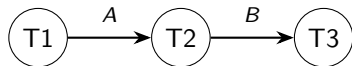
This is the lock-based counterpart of the read/write conflict definition: an exclusive lock corresponds to a write, a shared lock to a read.

Conflict Graph – Example

Schedule S_1 :

T1	T2	T3
write(A)		
	read(A)	
	write(B)	
		read(B)

Conflicts: $T1 \rightarrow T2$ on A , $T2 \rightarrow T3$ on B



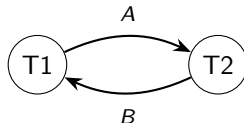
Acyclic – serializable.

Serial order: T1, T2, T3.

Schedule S_2 :

T1	T2
read(A)	
	read(B)
	write(A)
write(B)	

Conflicts: $T1 \rightarrow T2$ on A , $T2 \rightarrow T1$ on B



Cycle – not serializable.

Testing Serializability

Test

A schedule is conflict serializable if and only if its conflict graph is **acyclic**. A **topological sort** of the graph gives a valid serialization order.

Two strategies for guaranteeing serializability:

Post-hoc Generate a schedule, build its conflict graph, and test for cycles.

- ▶ Conceptually simple, but *inefficient* in practice: the full schedule must be collected before any check can be performed.

Protocol Impose a concurrency-control protocol that only ever *generates* serializable schedules – no cycle check needed at all.

- ▶ Standard approach in real systems (e.g. two-phase locking).

Locking Protocol

Delay transactions that might violate serializability by **locking** items while they are in use.

Shared lock $LS(Q)$ Grants *read* access to Q . Multiple transactions may hold a shared lock on Q simultaneously.

Exclusive lock $LX(Q)$ Grants *read and write* access to Q . At most one transaction may hold an exclusive lock on Q ; no shared locks may coexist with it.

Unlock $UN(Q)$ Releases any lock held on Q .

Waiting rule

A transaction must **wait** if the requested lock conflicts with a lock already held by another transaction.

Exception: if only shared locks exist on Q , a further $LS(Q)$ is granted immediately.

Two-Phase Locking (2PL)

The **two-phase locking protocol** divides each transaction's lifetime into exactly two phases:

Phase 1 – Growing

- ▶ May **acquire** locks.
- ▶ May **not release** locks.

Phase 2 – Shrinking

- ▶ May **release** locks.
- ▶ May **not acquire** locks.

The transition from phase 1 to phase 2 occurs the moment the **first lock is released**. From that point on, no new lock may be requested.

Guarantee

Any schedule produced by transactions that follow 2PL is **conflict serializable**.

2PL With Lock Conversion

Basic 2PL can be extended to allow **lock upgrades and downgrades** within the respective phase:

Phase 1 – Growing

- ▶ Acquire a shared lock $LS(Q)$.
- ▶ Acquire an exclusive lock $LX(Q)$.
- ▶ **Upgrade:** convert $LS(Q)$ to $LX(Q)$.

Phase 2 – Shrinking

- ▶ Release a shared lock $LS(Q)$.
- ▶ Release an exclusive lock $LX(Q)$.
- ▶ **Downgrade:** convert $LX(Q)$ to $LS(Q)$.

Guarantee

Lock conversion does not break the 2PL invariant. The protocol still assures **conflict serializability**.

Practical 2PL – System-Managed Locking

The transaction issues plain read/write. The system handles all lock operations transparently.

read D :

1. If T already holds *any* lock on D : perform read.
2. Else: wait until no other transaction holds an X-lock on D .
3. Grant an S-lock on D to T , perform read.

write D :

1. If T already holds an X-lock on D : perform write.
2. Else: wait until no other transaction holds *any* lock on D .
3. If T holds an S-lock on D : upgrade to X-lock.
Else: grant X-lock on D to T .
4. Perform write.

Lock release

Locks are released as late as safely possible. Two common policies are **strict 2PL** (release X-locks only at commit/abort) and **rigorous 2PL** (release *all* locks only at commit/abort) – discussed later.

Cascading Rollbacks

The problem

If a transaction T_1 aborts, every transaction that read data written by T_1 must also abort – and any transaction that read *their* writes must abort too, and so on. One abort can trigger a **cascade** of rollbacks.

The root cause is **dirty reads**:

Dirty data Data written by a transaction that has *not yet committed*. Its final value is uncertain – the writer may still abort.

Dirty read Reading dirty data written by another transaction. If the writer later aborts, the reader has used data that never officially existed.

Solution

Prevent transactions from reading dirty data at all. This eliminates cascading rollbacks because a reader can only depend on values that are already **permanent**.

Strict vs. Rigorous 2PL

Both variants tighten *when* locks are released to prevent cascading rollbacks.

Strict 2PL

X-locks are held until commit or abort. S-locks may be released earlier.

Effect: no dirty reads $\Rightarrow$ no cascading rollbacks.

Trade-off: early S-lock release allows more concurrency than rigorous 2PL.

Both variants satisfy the 2PL invariant and guarantee **conflict serializability**.

Rigorous 2PL

All locks (S and X) are held until commit or abort.

Effect: transactions appear to execute in strict commit order – strong isolation.

Trade-off: less concurrency; S-locks held to the end.

Aggressive vs. Conservative Protocols

Aggressive

Proceed as fast as possible, even if the situation may lead to an abort later.

Example: the practical 2PL lock-conversion protocol – locks are acquired on demand and released as soon as permitted.

Maximises concurrency, but aborts (and re-execution) are possible when conflicts are detected late.

Conservative

Do not perform work that may later have to be undone.

Example: each transaction requests *all* locks it will ever need at the start and holds them until it ends.

Advantages:

- ▶ **No deadlocks** – all locks are granted before work begins.
- ▶ **No starvation** – lock requests are queued fairly (FIFO).

Disadvantage: more waiting, lower concurrency.

Timestamp Protocol – Basic Idea

An alternative to locking: enforce serializability by requiring all accesses to respect **timestamp order**.

Timestamps Each transaction receives a unique, monotonically increasing integer timestamp on entry. This defines the target serial order.

Per-item state The system tracks two values for each item Q :

- ▶ $r\text{-max}(Q)$ – timestamp of the latest transaction that *read* Q .
- ▶ $w\text{-max}(Q)$ – timestamp of the latest transaction that *wrote* Q .

Access rules for transaction T with timestamp $ts(T)$

- ▶ **Read Q :** permitted only if $w\text{-max}(Q) \leq ts(T)$.
Otherwise: abort T and restart with a new (later) timestamp.
- ▶ **Write Q :** permitted only if $r\text{-max}(Q) \leq ts(T)$ and $w\text{-max}(Q) \leq ts(T)$.
Otherwise: abort T and restart with a new (later) timestamp.

Timestamp Protocol

Let T be the timestamp of the executing transaction.

read X:

1. **If** $T \geq w\text{-max}(X)$:
 - ▶ Perform read.
 - ▶ $r\text{-max}(X) \leftarrow \max(T, r\text{-max}(X))$
2. **Else: abort** T – trying to read a value already overwritten by a later transaction.

write X:

1. **If** $T \geq r\text{-max}(X)$ **and** $T \geq w\text{-max}(X)$:
 - ▶ Perform write;
 $w\text{-max}(X) \leftarrow T$
2. **Else if** $T < r\text{-max}(X)$:
 - ▶ **Abort** T – a later transaction already read the old value.
3. **Else** ($r\text{-max} \leq T < w\text{-max}$):
skip write (*Thomas Write Rule*).
 T continues; only this write is dropped – a later transaction's value supersedes it.

Strict Timestamp-Based Concurrency Control

To avoid cascading rollbacks, transactions read only **committed** values. Same rules as before, but with a *wait* step before acting.

read X:

1. **If** $T > w\text{-max}(X)$:
 - ▶ $r\text{-max}(X) \leftarrow \max(T, r\text{-max}(X))$
 - ▶ Wait until X is *committed*.
 - ▶ Perform read.
2. **Else** ($T \leq w\text{-max}(X)$):
abort T .
A later transaction already wrote X; waiting won't help.

write X:

1. **If** $T \geq r\text{-max}(X)$ **and** $T \geq w\text{-max}(X)$:
 - ▶ $w\text{-max}(X) \leftarrow T$
 - ▶ Wait until X is committed and pending reads on X are done.
 - ▶ Perform write.
2. **Else if** $T < r\text{-max}(X)$: **abort** T .
A later transaction already read the old value; cannot be fixed by waiting.
3. **Else: skip write** (Thomas Write Rule).
 T continues; only this write dropped.

The wait step ensures no transaction reads or overwrites *dirty* (uncommitted) data, eliminating cascading rollbacks.

SQL Isolation Levels

SQL defines four isolation levels that control which anomalies are permitted between concurrently executing transactions.

SET TRANSACTION ISOLATION LEVEL ...

SERIALIZABLE An equivalent serial execution schedule exists. Fully satisfies the ACID isolation requirement. No anomalies permitted.

REPEATABLE READ No dirty reads, no unrepeatable reads. Phantom reads may occur (new rows inserted by another transaction become visible).

READ COMMITTED No dirty reads. Unrepeatable reads and phantoms may occur. Most common default in practice (e.g. PostgreSQL, Oracle).

READ UNCOMMITTED Dirty reads are permitted. Highest concurrency, lowest isolation.

Choosing the right level is an application decision: stronger isolation costs more in concurrency and performance.

Isolation Levels – How They Differ

Each level below **SERIALIZABLE** relaxes one or more constraints, trading isolation for concurrency.

REPEATABLE READ Row-level **read locks are held** until end of transaction – a re-read always returns the same value. However, *new* rows inserted by another committed transaction *can* appear between two reads of the same range (**phantom reads**).

READ COMMITTED Read locks are **acquired but released immediately** after each read. No dirty reads, but the same row can return different values on re-read (**unrepeatable reads**), and phantoms are also possible.

READ UNCOMMITTED **No read locks at all.** Dirty reads are possible – a row may change value or vanish between reads because it was written by an uncommitted transaction.

Problems with Serializability

Both main approaches to enforcing serializability carry overhead.

Locking

- ▶ Long waits – especially when locks depend on end-user interaction.
- ▶ Transactions may be aborted due to **deadlock**.
- ▶ Significant bookkeeping overhead for the lock manager.

Optimistic / Timestamps

- ▶ Potentially many **aborted and restarted** transactions when conflicts are detected late.
- ▶ Large overhead for bookkeeping and **validation** at commit time.

Neither approach is universally best – the right choice depends on the workload's conflict rate and transaction duration.

Transactions and Constraints

Normal behaviour

Integrity constraints are checked **after each modification**. If a constraint is violated, the transaction is **rolled back**.

Sometimes a transaction must **temporarily violate** a constraint mid-execution – for example, swapping two unique values requires both to be in an invalid state for one step.

Deferred constraints

A constraint can be declared **deferrable**:

```
DEFERRABLE INITIALLY DEFERRED
```

The check is then postponed until **commit time**. If the constraint is violated at commit, the transaction is rolled back as usual.

Read-Only Transactions

A transaction that only reads data **can never violate serializability** – it produces no writes, so no conflict ordering is imposed on other transactions.

This can be declared explicitly in SQL:

```
SET TRANSACTION READ ONLY;
```

(A DBMS generally cannot infer which statements a transaction will execute ahead of time, so the declaration is needed.)

Benefit

Other transactions **never need to wait** for a read-only transaction to finish. The DBMS can serve it without acquiring write locks or inserting it into conflict-graph checks.